

Python для обчислювальних задач

Лабораторна робота №1

60 хвилин пояснення. Python. JupyterLab.



Ні, зроби саме завдання 6.1, варіант 7 з коментарями

Ось програмний код для розв'язання Лабораторної роботи № 6, завдання 6.1 (варіант 7).

Відповідно до умов сьомого варіанта: PDF

- Функція: $f(x) = \left(1 - \frac{1}{x^2}\right) \cos(x)$
- Інтервал: $[-2; -1]$
- Точка $x_0 = -2$
- Формула для розрахунку $f'(x)$: 1 (Несиметрична пряма, 1-го порядку: $f'(x_i) = \frac{f(x_{i+1}) - f(x_i)}{h}$) PDF
- Формула для розрахунку $f''(x)$: 4 (Несиметрична обернена, 2-го порядку: $f''(x_i) = \frac{2f(x_i) - 5f(x_{i-1}) + 4f(x_{i-2}) - f(x_{i-3}))}{h^2}$) PDF

Програмний код з детальними коментарями:

Python ⬇️ 📄

```
import sympy as smp
import numpy as np
import matplotlib.pyplot as plt
import math

# =====
# 1. Аналітичне диференціювання за допомогою SymPy
# =====

# Задаємо символічну змінну
x = smp.Symbol('x')

# Задаємо символічну функцію f(x)
f = (1 - 1 / x**2) * smp.cos(x)
print("f(x) =", f)

# Точка, в якій обчислюються похідні
x0_val = -2

# Одержуємо аналітичний вираз першої похідної
f1 = smp.diff(f, x, 1)
```

Лабораторна робота № 1
ПРОГРАМУВАННЯ МОВОЮ *PYTHON*. ІНТЕРПРЕТАТОР *PYTHON* ЯК
КАЛЬКУЛЯТОР. РОЗГАЛУЖЕНІ І ЦИКЛІЧНІ ПРОГРАМИ

Мета роботи: ознайомлення з інтерфейсом і можливостями інтегрованого середовища розробки *JupyterLab*, з основними числовими типами даних мови *Python*. Одержання практичних навичок використання умовних операторів, організації циклів і написання програм для обчислення математичних виразів.

1.1 Інсталяція, запуск і початок роботи в *JupyterLab*

В рамках навчального курсу для зручності і наочності пропонується використання середовища розробки *JupyterLab*, що є програмою-блокнотом для запису і запуску програмного коду в *Python*. Для роботи рекомендується попередньо встановити графічний інтерфейс *Anaconda Navigator* [1].

Найпростіший і найбільш контрольований спосіб встановлення Python на Windows — використання офіційного інстальатора.

1 Завантажте інстальатор

Офіційний сайт

Перейдіть на python.org/downloads та завантажте рекомендовану стабільну версію Windows Installer (зазвичай 64-bit).

Перевірка: Файл із розширенням .exe з'явиться в папці «Завантаження».

2 Запустіть встановлення та увімкніть PATH

Критичний крок

Запустіть завантажений файл.

У першому ж вікні інстальатора обов'язково поставте прапорець «Add python.exe to PATH» унизу. Після цього натисніть «Install Now» (або «Customize installation», якщо потрібно змінити директорію встановлення чи увімкнути встановлення для всіх користувачів).

Перевірка: Процес завершиться повідомленням «Setup was successful». За потреби натисніть «Disable path length limit», якщо інстальатор запропонує це на фінальному екрані.

3 Перевірте коректність у терміналі

CLI

Відкрийте термінал (PowerShell, Command Prompt або Windows Terminal) і виконайте команди:

```
PowerShell

python --version
pip --version
```

Перевірка: Команди повернуть версію інтерпретатора (наприклад, Python 3.12.x) та пакетного менеджера pip. Якщо з'являється помилка «command not found», це означає, що прапорець Add to PATH не було встановлено (потрібно повторно запустити інстальатор у режимі Modify/Repair).

Головна концептуальна різниця — Python 3 відмовився від зворотної сумісності з Python 2 заради виправлення фундаментальних вад мови (насамперед у роботі з текстом/кодуваннями та узгодженості типів). Підтримка Python 2 повністю припинена з 1 січня 2020 року.

Ключові технічні відмінності:

Характеристика	Python 2	Python 3
<code>print</code>	Оператор (інструкція): <code>print "text"</code>	Функція: <code>print("text")</code>
Рядки та байти	<code>str</code> — це ASCII-байти, <code>unicode</code> — окремий тип	<code>str</code> — це виключно Unicode (текст), для сирих байтів є тип <code>bytes</code>
Цілочисельне ділення	<code>5 / 2</code> повертає <code>2</code> (truncation)	<code>5 / 2</code> повертає <code>2.5</code> (float). Для цілочисельного є оператор <code>//</code>
Генератори та ітератори	Функції на кшталт <code>range()</code> , <code>map()</code> , <code>filter()</code> , <code>.keys()</code> повертали <code>list</code> у пам'ять	Повертають генератори/view-об'єкти (<code>range</code> , <code>dict_keys</code>), пам'ять не витрачається наперед
Порівняння несумісних типів	Дозволялося: <code>'abc' > 5</code> повертало <code>True</code>	Викликає <code>TypeError</code>
Типи цілих чисел	Було два типи: <code>int</code> (32/64 біти) та <code>long</code> (довільна точність)	Тільки один уніфікований тип <code>int</code> із необмеженою точністю
Обробка винятків	Синтаксис <code>except Exception, e:</code>	Синтаксис <code>except Exception as e:</code>
Введення даних	<code>raw_input()</code> (повертає рядок), <code>input()</code> (виконував <code>eval()</code>)	Тільки безпечний <code>input()</code> (повертає <code>str</code>)

Anaconda — це дистрибутив мов Python та R, орієнтований на Data Science, наукові обчислення та ML. Головна його мета — спростити менеджмент залежностей і середовищ.

Ключові задачі, які вона вирішує:

- **Менеджер пакетів `conda` замість `pip`:**
`pip` здебільшого встановлює Python-пакети. `conda` — це системно-агностичний пакетний менеджер, який встановлює не лише Python-код, а й бінарні не-Python залежності (бібліотеки C/C++, CUDA, cuDNN, OpenBLAS, HDF5 тощо), не вимагаючи локальної компіляції чи ручного налаштування компіляторів у системі.
- **Ізоляція середовищ:**
Дозволяє створювати віртуальні оточення з різними версіями самого інтерпретатора Python (на відміну від стандартного `venv`, який прив'язаний до поточної версії системного Python).
- **Готовий науковий стек (Out-of-the-box):**
Базовий дистрибутив містить понад 250 попередньо протестованих на сумісність бібліотек (NumPy, SciPy, Pandas, Matplotlib, Scikit-learn) та інструменти (Jupyter Notebook/Lab, Spyder).

Коли доцільно обирати:

- Складні наукові проєкти або Data Science / ML, де потрібні низькорівневі C/CUDA оптимізації без складання коду з вихідників.
- Потрібно швидко розгорнути ідентичне середовище на Windows, macOS чи Linux без клопоту з системними драйверами та бібліотеками.

JupyterLab — це сучасне інтерактивне вебове середовище розробки (IDE) для роботи з блокнотами Jupyter (`.ipynb`), кодом і даними. Воно є наступником класичного інтерфейсу Jupyter Notebook.

Ключові особливості та можливості:

- **Модульний інтерфейс з вкладками (Tabs & Docking):**
На відміну від старого Jupyter Notebook (де кожен блокнот відкривався в окремій вкладці браузера), JupyterLab має єдиний робочий простір. Можна розташовувати блокноти, текстові редактори, термінали та вікна перегляду графіків поруч у спліт-режимі (drag-and-drop).
- **Універсальність компонентів:**
В одному середовищі доступні:
 - Блокноти Jupyter (`.ipynb`) з інтерактивним виконанням комірок і збереженням стану виводу.
 - Текстові редактори з підсвічуванням синтаксису (для `.py` , `.md` , `.json` , `.csv` тощо).
 - Вбудований повнофункціональний системний термінал.
 - Інтерактивні консолі (Python/IPython), прив'язані безпосередньо до активного ядра блокнота (зручно для тестування однорядкового коду без додавання комірок).
- **Багатомовність (Kernels):**
Хоча найчастіше використовується з Python, завдяки архітектурі ядер Jupyter підтримує виконання коду на R, Julia, C++, Scala тощо.
- **Розширюваність (Extensions):**
Архітектура повністю побудована на плагінах. Підтримує розширення для інтеграції з Git, візуального дебагінгу (інтерактивний налагоджувач змінних і точок зупину), підтримки мовних серверів (LSP — автодоповнення, сигнатури функцій) та спільної роботи в реальному часі (Real-Time Collaboration).
- **Робота з форматами даних:**
Має вбудовані засоби перегляду для зображень, інтерактивних HTML-віджетів, Markdown із рендерингом LaTeX-формул, геоданих (GeoJSON) і великих таблиць CSV/TSV.

JupyterLab і числові дані

Notebook поєднує код, пояснення та результати.

Kernel зберігає стан між запусками комірок.

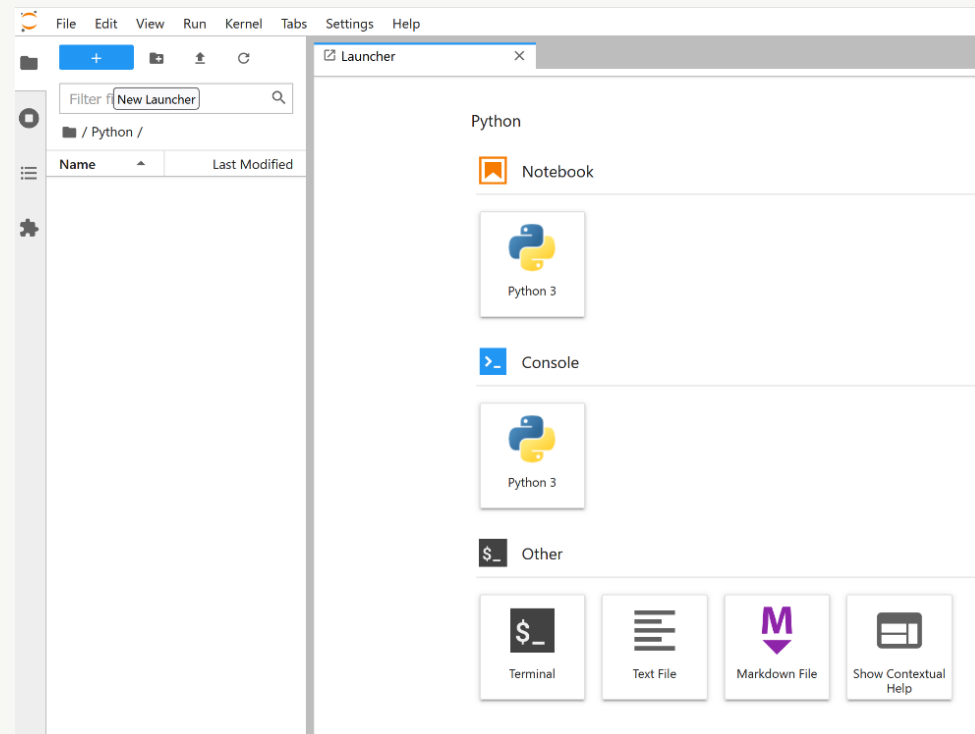
Порядок виконання має бути відтворюваним.

JupyterLab: перший робочий notebook

Launcher / Notebook / Python.

Назва: Lab_1_Прізвище.ipynb;
варіант у текстовій комірці.

Код виконуємо комірками;
пояснення пишемо в
Markdown.



Інтерфейс із посібника 6619, с. 9

Визначення: ім'я, значення і тип

Присвоєння зв'язує ім'я з об'єктом: `x = 1.45`.

`int`: цілі числа; `float`: наближені дійсні; `complex`: комплексні.

`str` з цифрами залишається текстом; `type(x)` показує тип.

```
x = 1.45
z = 2 + 3j
print(type(x), abs(z))
print(12 + 3)
print("12" + "3")
```

Навчальний фрагмент синтаксису

Прихований стан: чому старе число вижило

Подія	а	у
Обидві комірки виконано	2	3
Змінено лише а	10	3
Повторно обчислено у	10	11

Зміна а не перераховує у автоматично.

Практичне завдання: чистий запуск

Створіть notebook із назвою роботи та номером варіанта.

Окремо задайте дані, виконайте розрахунок і поясніть результат.

Змініть один параметр та повторіть запуск зверху вниз.

Результат: відтворюваний файл .ipynb.

Математичний вираз у Python

Дужки відтворюють структуру формули.

Степінь у Python: `**`; множення: `*`.

Довгий вираз зручно розділити на змістовні частини.

Оператори: три типові пастки

Вираз А	Значення А	Вираз Б	Значення Б
$-2^{**}2$	-4	$(-2)^{**}2$	4
$2^{**}3^{**}2$	512	$(2^{**}3)^{**}2$	64
$7 // 3$	2	$-7 // 3$	-3

**** означає степінь. Дужки явно задають групування.**

math: функції з чітким змістом

`import math`; виклик має вигляд
`math.sin(x)`.

`sin` і `cos` приймають радіани; π
беремо з `math.pi`.

`ln(x)`: `math.log(x)`; `lg(x)`:
`math.log10(x)`.

`math.sqrt(x)`, `math.exp(x)`,
`math.factorial(n)`.

```
import math

print(math.sin(math.pi / 2))
print(math.sin(2)**2)
print(math.sin(2**2))
print(math.log10(100))
```

Навчальний фрагмент синтаксису

Визначення: область допустимих даних

$$f(x) = \frac{\ln(x+1)}{x-2}, \quad D = (-1, 2) \cup (2, +\infty)$$

$D = \{x : \text{усі операції виразу визначені в обраній числовій моделі}\}.$

Для $\sqrt{u}$: $u \geq 0$; для $\ln(u)$: $u > 0$.

У знаменнику: $v \neq 0$.

Перевіряємо умови до виконання небезпечної операції.

Завдання 1.1: обчислення величин

$$a = \frac{2b \cos(x - \pi/6)}{1/2 + \sin^2 y}, \quad m = 1 + \frac{z^2}{3 + y^2/5}$$

За своїм варіантом перенесіть усі формули зі с. 19–21.

Задайте дані окремо та перевірте допустимість операцій.

Виведіть імена величин і результати; додайте ручну перевірку.

Спільний приклад: варіант 1, $x = 1,45$; $y = -1,22$; $z = 3,5$; $b = 3,07$.

Одна програма, кілька правил

Спочатку визначаємо, яке правило діє для x .

Обчислюємо тільки вибрану гілку.

Межа інтервалу належить рівно одній гілці.

Визначення: кусочно задана функція

$$y(x) = \begin{cases} ax + b \cos x, & 0 \leq x < 0.5, \\ bx^2 + c \sin(2x), & 0.5 \leq x \leq 1. \end{cases}$$

Демонстраційний варіант 1: $a = 0.75$, $b = 1.19$, $c = -2.5$.

Область завдання: $0 \leq x \leq 1$.

Точка $x = 0.5$ належить другій гілці.

if, elif, else: читаємо шлях виконання

Перевірки виконуються зверху вниз.

Виконується перша істинна гілка.

Відступи показують, які рядки належать гілці.

```
x = 0
if x < 0:
    print("від'ємне")
elif x == 0:
    print("нуль")
else:
    print("додатне")
```

Навчальний фрагмент синтаксису

Межа часто знаходить баг швидше

х	Очікувана дія
-0.1; 1.1	Повідомлення: поза [0; 1]
0; 0.49	Перша формула
0.5; 0.51; 1	Друга формула

На межі $x = 0.5$ діє друга формула.

Python Challenge: одна точка, одна гілка

Вхід: x та коефіцієнти вашого варіанта.

Вихід: x , номер гілки, значення y або пояснення відмови.

Тести: усі межі й по точці з кожного інтервалу.

Підказка: відокремте перевірку області від вибору формули.

Цикл повторює перевірений крок

Одна ітерація обробляє один аргумент.

Алгоритм задає однозначні кроки для класу вхідних даних.

Скінченний список вузлів гарантує завершення обходу.

Визначення: рівномірна сітка

$$m = \frac{R - L}{h} \in \mathbb{N}, \quad x_i = L + ih, \quad i = 0, \dots, m$$

$h > 0$; $m = (R - L)/h$ має бути цілим числом.

Відрізок із m кроків містить $m + 1$ вузлів.

Для $[0; 1]$, $h = 0.1$: індекси $0 \dots 10$, усього 11 вузлів.

for: читаємо цикл без ручного копіювання

`range(4)` дає індекси 0, 1, 2, 3.

Тіло циклу позначають відступами.

`break` завершує цикл; `continue` пропускає решту ітерації.

```
for i in range(4):  
    y = i ** 2  
    print(i, y)  
  
# Перший елемент списку:  
values = [10, 20, 30]  
print(values[0])
```

Навчальний фрагмент синтаксису

NumPy: крок або кількість вузлів

`arange` задає крок; дробовий крок потребує обережності.

`linspace` задає кількість; кінці включено за замовченням.

Для базового рішення достатньо `range` та обчислення `x` за `i`.

```
import numpy as np

# Третій аргумент — крок:
np.arange(2, 6, 1)

# Третій аргумент — кількість:
np.linspace(2, 3, 5)
```

Навчальний фрагмент синтаксису

Python Challenge: завдання 1.2

Демонстраційний варіант 1: $x \in [0; 1]$, $h = 0.1$.

Коефіцієнти: $a = 0.75$, $b = 1.19$, $c = -2.5$.

Вихід: 11 рядків зі стовпцями i , x , гілка, y .

Контроль: обидва кінці та $x = 0.5$; перевірка кожної гілки.

Звіт, який можна відтворити

Постановка й номер призначеного варіанта.

Вхідні дані, формули, код і підписані результати.

Перевірки меж, кількості вузлів та числових значень.

Повторний запуск усіх комірок після очищення стану.

Самостійна робота після пояснення

Створіть notebook, який виконується після перезапуску kernel.

Виконайте завдання 1.1 для призначеного варіанта.

Перевірте всі гілки завдання 1.2 на контрольних аргументах.

Побудуйте повну таблицю та додайте висновок про перевірки.

Від формули до перевіреної таблиці

Формула потребує правильного синтаксису й області визначення.

Умова вибирає правило, цикл повторює його на сітці.

Перевірки та повторний запуск роблять результат відтворюваним.